

Arquitetura C4 do NovaCore

Guia visual para alunos | Arquitetura implementada + evolução resumida

Do processo à força de trabalho

Control Tower analisa o incidente. Control Plane mede os agentes. Maestro organiza propostas de melhoria. Segundo Cérebro preserva conhecimento revisado.

Como ler este material

C1 apresenta pessoas e sistemas. C2 mostra aplicações, adaptadores e armazenamento. C3 abre componentes internos. Implantação e fluxos são vistas complementares, não novos níveis C4.

O que esta edição representa

Código consolidado em main no commit `c5ede1a75f41af8a01cdb0e9fd16d9a7836d0f3f`. A documentação descreve o LAB local existente, sem inventar infraestrutura de produção ou ações autônomas.

Formato de uso

Cada diagrama tem uma página de leitura guiada: ordem de explicação, limites e arquivos a consultar. As mesmas relações estão disponíveis em Mermaid, SVG e no modelo JSON editável.

Legenda e índice

Um container C4 é uma aplicação ou armazenamento; não é necessariamente um container Docker.

 Pessoa / papel

 Sistema em foco

 Container lógico

 Componente

 Armazenamento

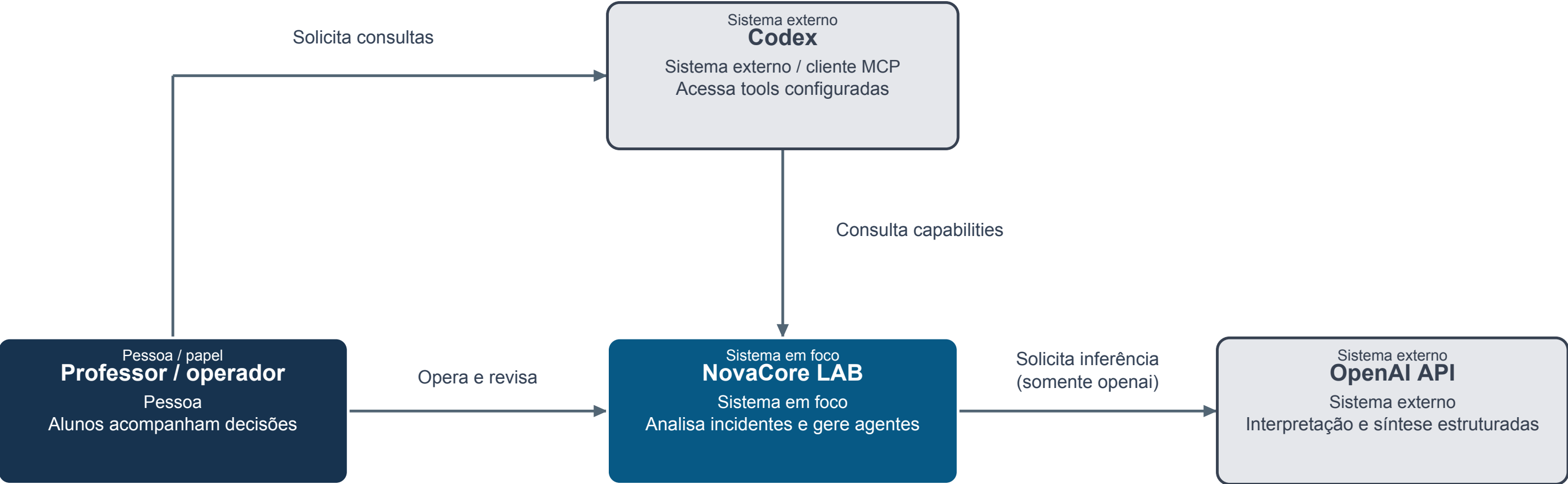
 Sistema externo

Setas indicam chamadas, dependências ou passos explicitamente rotulados. Respostas de request retornam ao chamador e não exigem uma segunda seta. Contornos tracejados delimitam o escopo interno quando aplicável.

- 01 | C1 · Contexto | Quem usa e de quem depende?
- 02 | C2 · Containers / vista operacional | Como um incidente é executado?
- 03 | C2 · Containers / vista de gestão | Onde ficam UI, gestão e memória?
- 04 | Complemento · Fronteira MCP | Outra fronteira, os mesmos serviços
- 05 | C3 · Componentes do worker | O que existe dentro de um worker?
- 06 | C3 · Componentes da API / consultas | Medir antes de recomendar
- 07 | C3 · Componentes da API / gestão | Síntese, proposta e conhecimento
- 08 | Complemento · Implantação local | Como isso roda no computador do professor?
- 09 | Complemento · Fluxo dinâmico | Do pedido ao resultado durável
- 10 | Complemento · Fluxo dinâmico | Da proposta ao contexto reutilizável
- 11 | Complemento · Evolução pedagógica | Um sistema, quatro necessidades

Quem usa e de quem depende?

C1 · Contexto | Sistema NovaCore: Control Tower + Control Plane



Leitura guiada | Quem usa e de quem depende?

Use depois do diagrama: explicar o porquê, não apenas os nomes das tecnologias.

COMO EXPLICAR

1. Comece pelo ator: o professor opera e revisa; os alunos observam arquitetura e limites.
2. NovaCore reúne análise do processo industrial e gestão da workforce. Codex é um cliente externo; não é o Maestro.

LIMITES IMPORTANTES

1. Não há integração real com ERP, fornecedor, transporte ou pagamento. O case utiliza arquivos locais e dados sintéticos.
2. Mock dispensa chamadas ao provider. A eventual inferência própria do cliente Codex está fora do escopo do backend NovaCore.

ONDE ENCONTRAR NO CÓDIGO

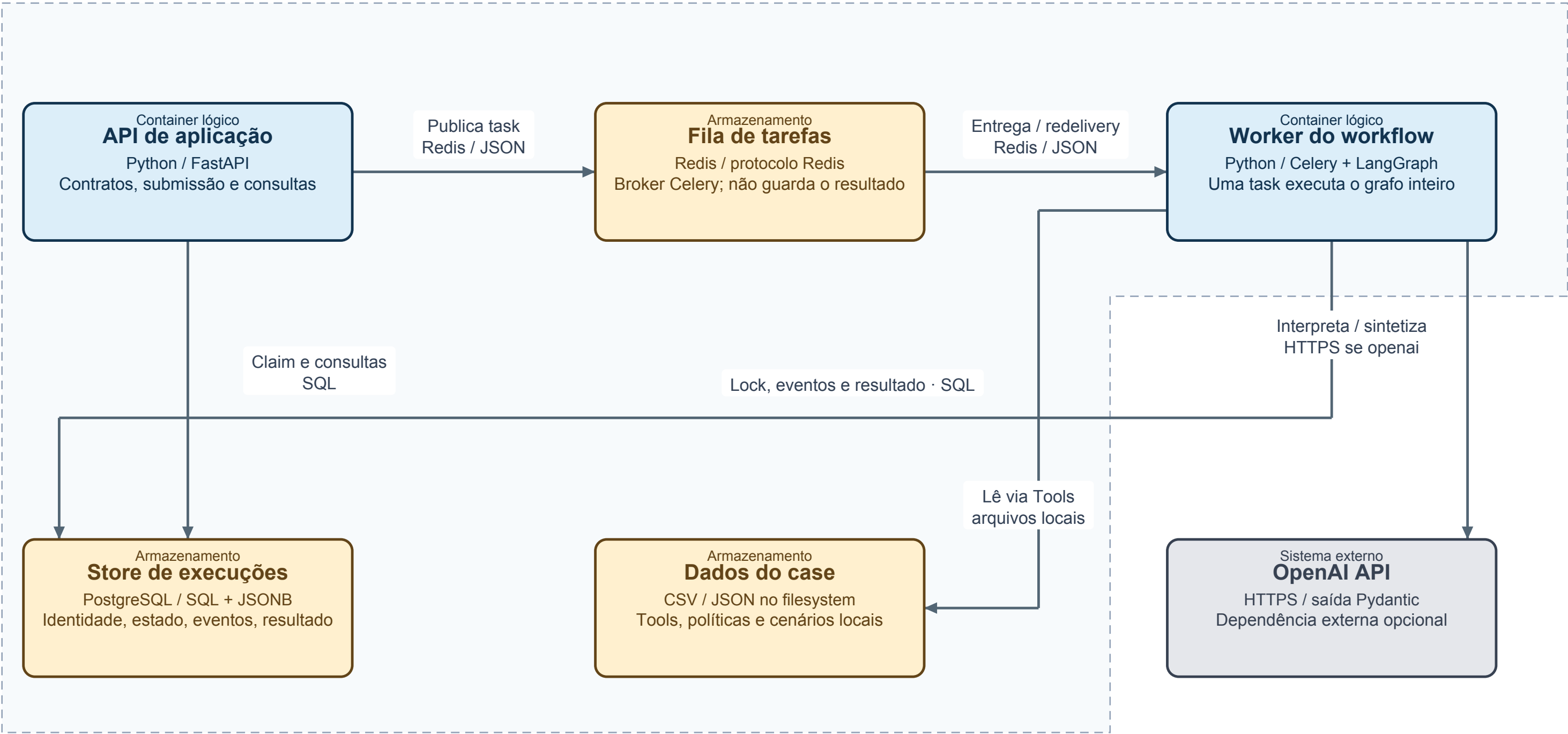
`src/control_tower/application.py`

`src/control_tower/mcp/server.py`

`src/control_tower/cockpit/api.py`

Como um incidente é executado?

C2 · Containers / vista operacional | Containers lógicos do caminho HTTP; MCP detalhado na vista 04



Leitura guiada | Como um incidente é executado?

Use depois do diagrama: explicar o porquê, não apenas os nomes das tecnologias.

COMO EXPLICAR

1. A API confirma a publicação e devolve `execution_id`. O trabalho acontece no worker, não no request de submissão.
2. PostgreSQL é a fonte do resultado. Redis não é result backend: `task_ignore_result=True`.

LIMITES IMPORTANTES

1. O claim é persistido antes da publicação: existe uma janela de falha entre banco e broker. O LAB usa reenvio explícito da mesma identidade; não implementa transactional outbox.
2. CLI distribuída chama o mesmo producer/store por outro processo Python. A CLI local da Aula 1 executa o grafo diretamente, sem fila.

ONDE ENCONTRAR NO CÓDIGO

`src/control_tower/distributed/producer.py`

`src/control_tower/distributed/tasks.py`

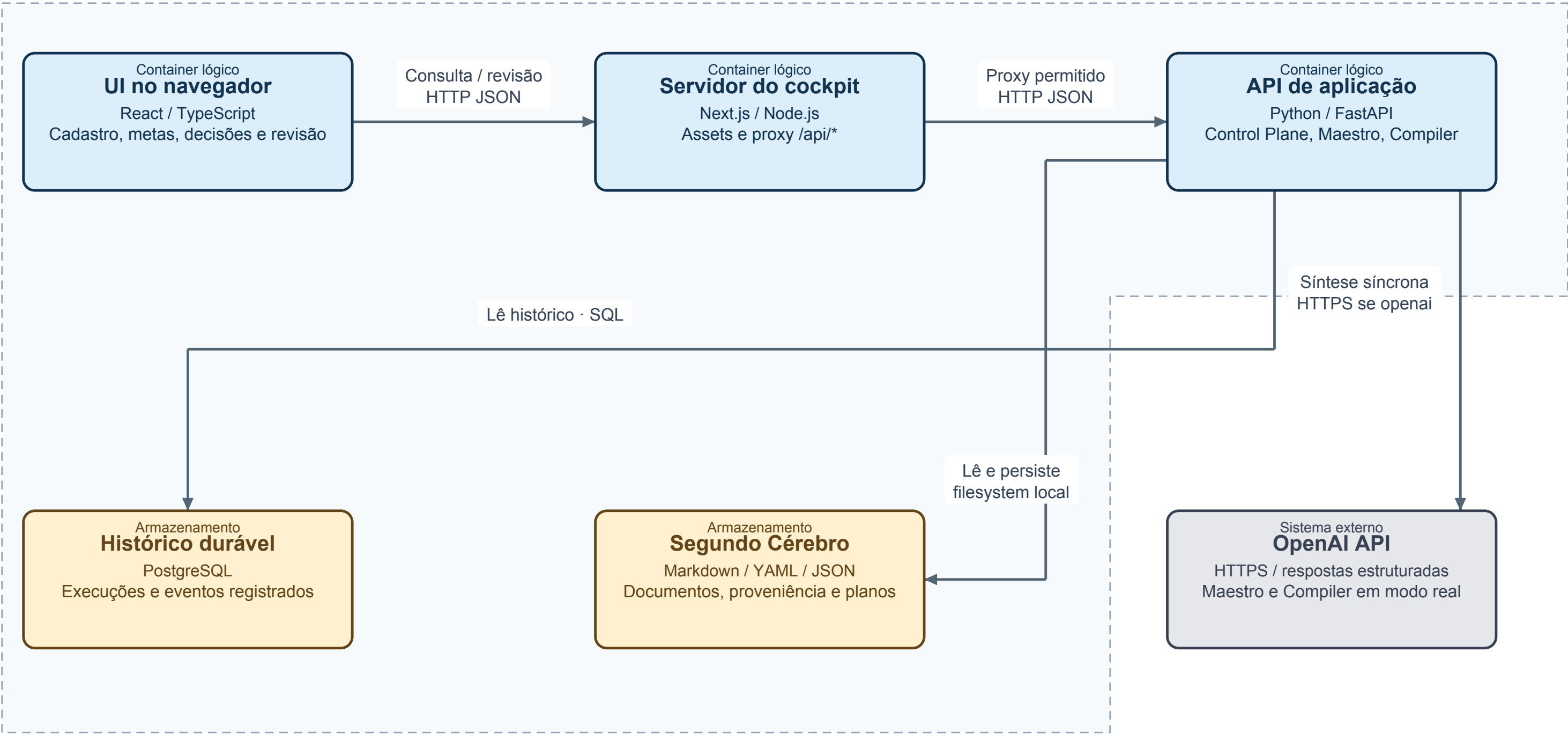
`src/control_tower/distributed/store.py`

`src/control_tower/distributed/celery_app.py`

`src/control_tower/main.py`

Onde ficam UI, gestão e memória?

C2 · Containers / vista de gestão | Cockpit e armazenamento; API é a mesma da vista operacional



Leitura guiada | Onde ficam UI, gestão e memória?

Use depois do diagrama: explicar o porquê, não apenas os nomes das tecnologias.

COMO EXPLICAR

1. A UI não calcula metas ou custos. O proxy encaminha somente cockpit, maestro e knowledge; o backend valida as operações.
2. Conversa e extração são chamadas síncronas no processo servidor. Não são tasks Celery nem jobs do LangGraph empresarial.

LIMITES IMPORTANTES

1. A fonte didactic lê fixtures e sinais sintéticos; durable lê o PostgreSQL. Uma não preenche silenciosamente lacunas da outra.
2. Sessões de chat ficam em memória do processo. Planos e conhecimento são arquivos persistidos em volume; não há vector database ou graph database.

ONDE ENCONTRAR NO CÓDIGO

`web/app/api/[...path]/route.ts`

`src/control_tower/cockpit/api.py`

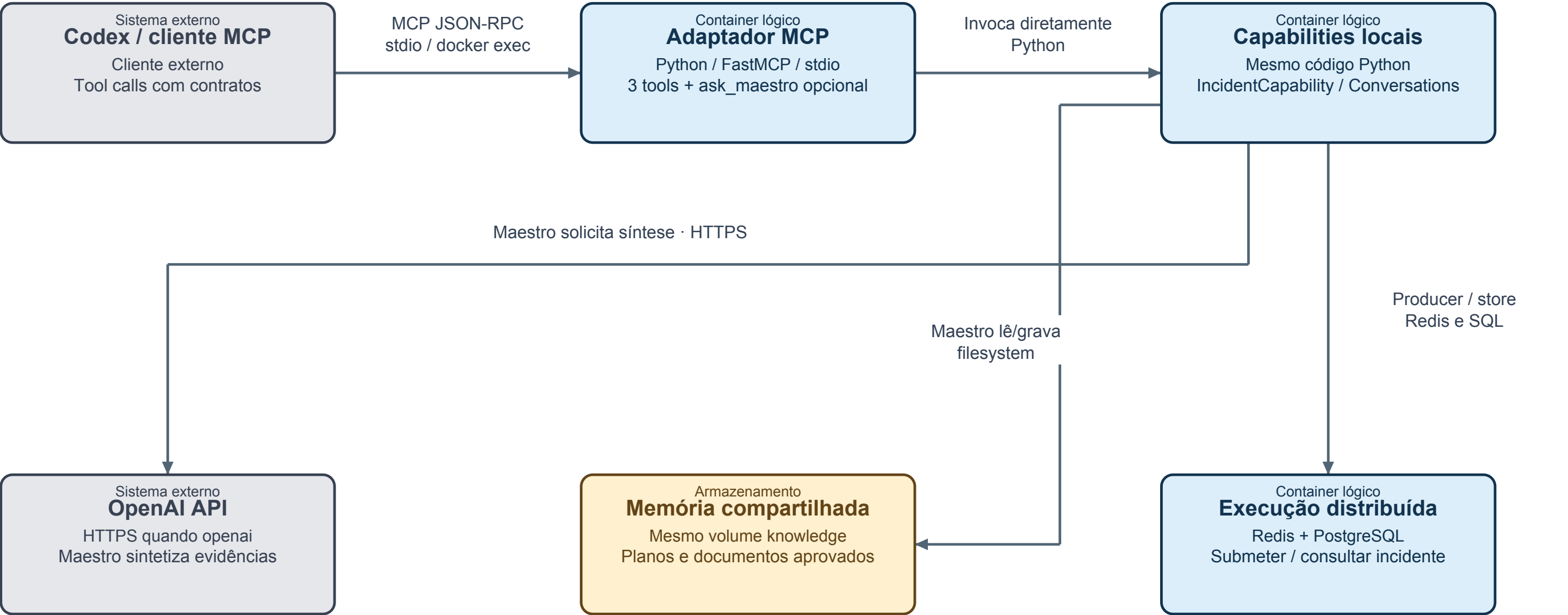
`src/control_tower/cockpit/presentation.py`

`src/control_tower/cockpit/knowledge.py`

`src/control_tower/cockpit/conversation.py`

Outra fronteira, os mesmos serviços

Complemento · Fronteira MCP | Processo Python transitório; sem novo serviço Compose nem porta MCP



Leitura guiada | Outra fronteira, os mesmos serviços

Use depois do diagrama: explicar o porquê, não apenas os nomes das tecnologias.

COMO EXPLICAR

1. O launcher executa `python -m control_tower.mcp.server --with-maestro` dentro do container api já iniciado. Isso cria outro processo, não uma chamada HTTP à API.
2. IncidentCapability e Conversations são reutilizados como código. O MCP possui sua própria instância de sessão; o chat do navegador não é compartilhado.

LIMITES IMPORTANTES

1. `submit_incident`, `get_execution_status` e `get_execution_result` permanecem disponíveis. `ask_maestro` é adicionada pelo launcher opcional.
2. MCP não oferece tool de aprovar conhecimento, mudar lifecycle ou executar plano. A memória persistida é comum; a autoridade não aumenta com a troca de cliente.

ONDE ENCONTRAR NO CÓDIGO

`scripts/start_maestro_mcp.sh`

`scripts/start_lesson04_mcp.sh`

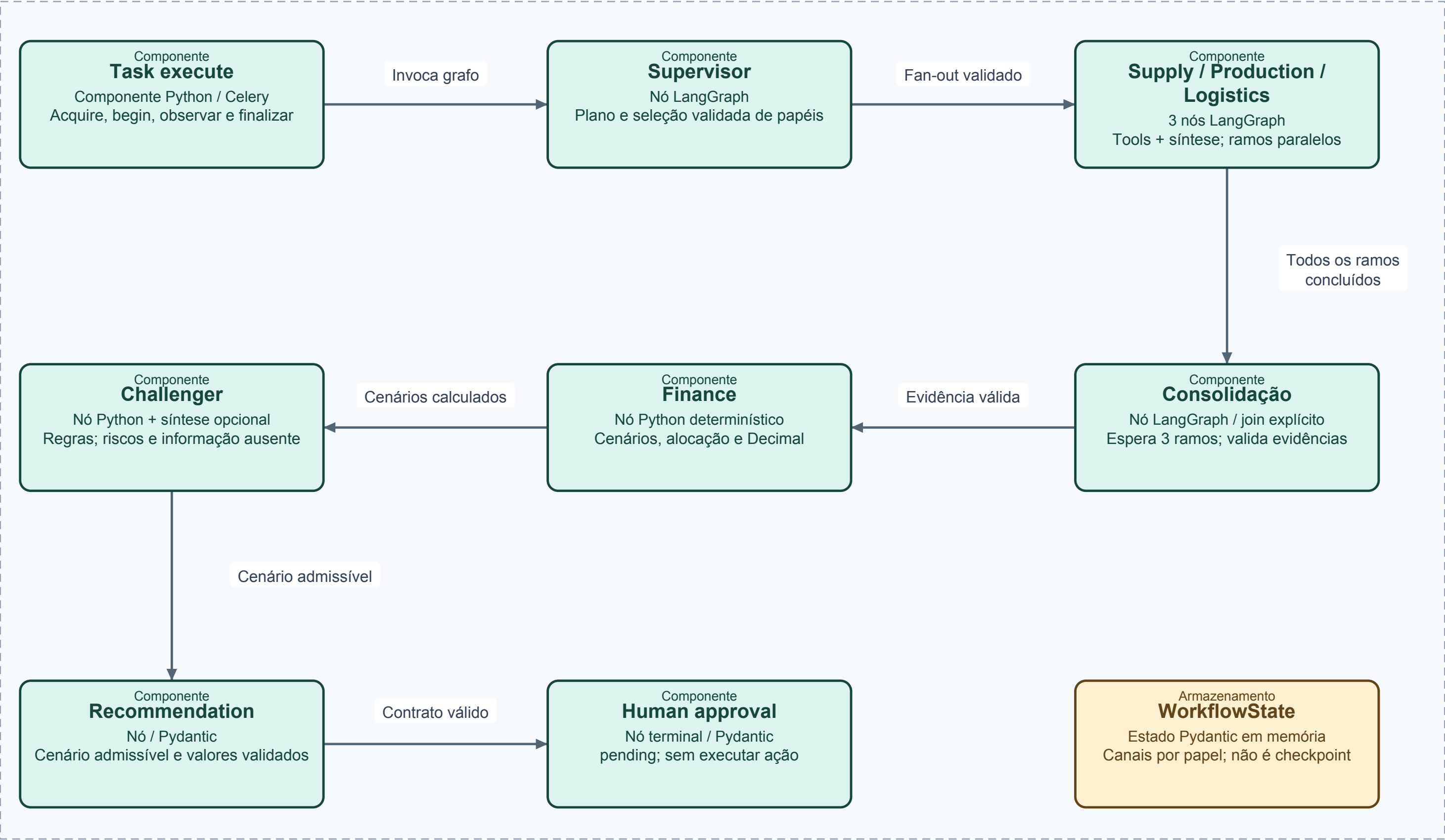
`src/control_tower/mcp/server.py`

`src/control_tower/mcp/tools.py`

`src/control_tower/mcp/maestro.py`

O que existe dentro de um worker?

C3 · Componentes do worker | Processo Python que executa a task Celery; nós não são microserviços



Leitura guiada | O que existe dentro de um worker?

Use depois do diagrama: explicar o porquê, não apenas os nomes das tecnologias.

COMO EXPLICAR

1. Celery distribui execuções completas entre workers. LangGraph coordena os papéis dentro de uma execução. O paralelismo local dos especialistas é outra dimensão.
2. O join aguarda todos os três ramos, não a primeira resposta. Cada especialista atualiza um canal próprio do estado.

LIMITES IMPORTANTES

1. Esta vista mostra o caminho de sucesso. Supervisor, consolidação, Finance, Challenger e Recommendation podem levar a blocked. Um especialista com erro é verificado no join.
2. OpenAI adiciona interpretação e sínteses nos mesmos nós. Finance, tools, políticas e validação permanecem determinísticos. Approval termina o grafo; não há interrupt/resume persistente.

ONDE ENCONTRAR NO CÓDIGO

`src/control_tower/distributed/tasks.py`

`src/control_tower/graph/workflow.py`

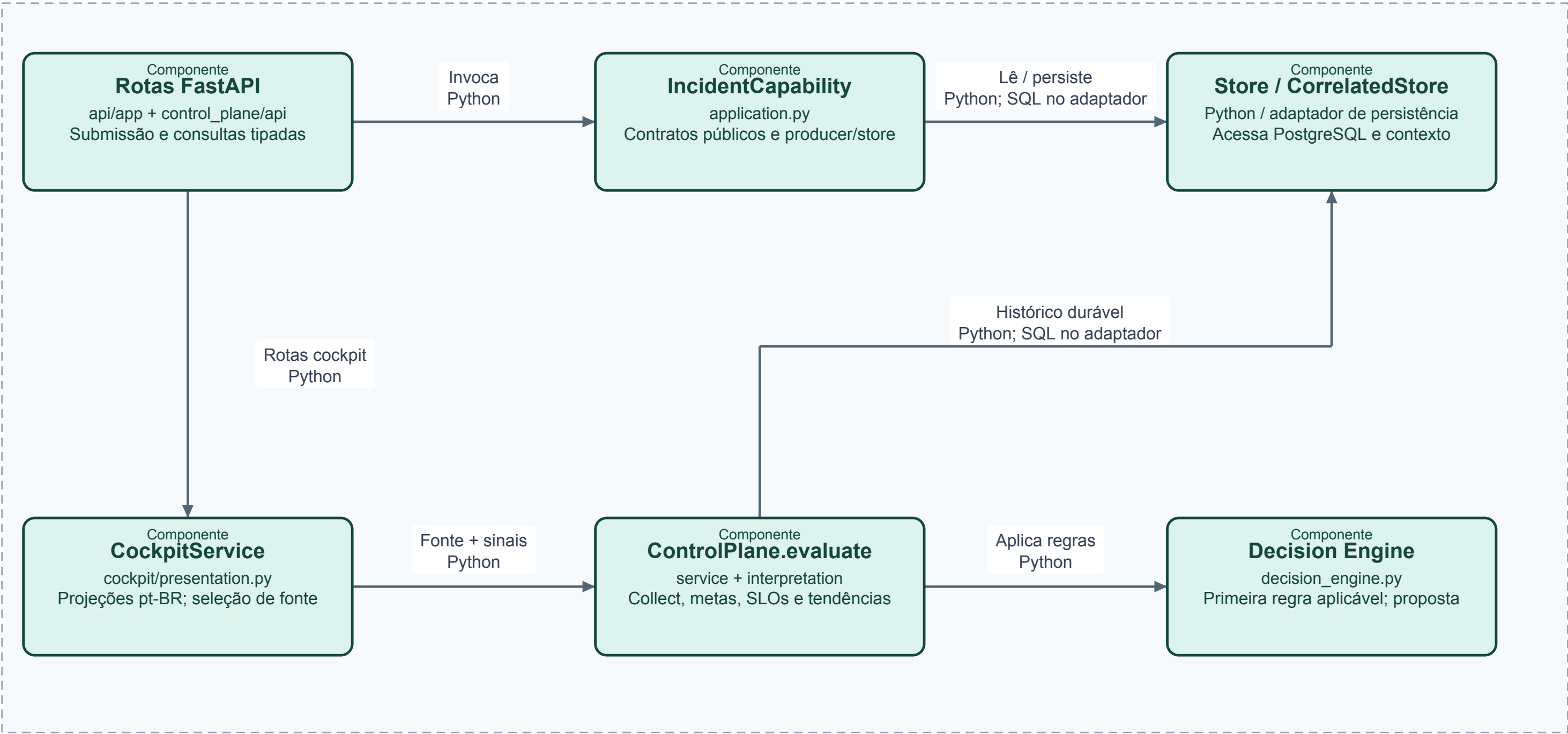
`src/control_tower/graph/state.py`

`src/control_tower/agents/finance.py`

`src/control_tower/agents/interpretation.py`

Medir antes de recomendar

C3 · Componentes da API / consultas | Módulos Python na API; não serviços implantáveis separados



Leitura guiada | Medir antes de recomendar

Use depois do diagrama: explicar o porquê, não apenas os nomes das tecnologias.

COMO EXPLICAR

1. O Control Plane transforma sinais em métricas e limites, e então em recomendação. A chamada consulta evidências; não atua sobre workers ou modelos.
2. Registry, pricing e configuração de SLOs vêm de código/arquivos locais. A fixture didática usa projeções próprias e mantém a origem explícita.

LIMITES IMPORTANTES

1. A seta representa chamada/dependência, não apenas trânsito de dados: resultados retornam ao chamador. Producer/Redis estão na vista C2 operacional.
2. Supply 74/90 mede cobertura didática de evidências. A meta técnica do Registry tem outro significado; os 50 exemplos de cobertura não são as 6 execuções ilustrativas.

ONDE ENCONTRAR NO CÓDIGO

`src/control_tower/api/app.py`

`src/control_tower/control_plane/api.py`

`src/control_tower/control_plane/service.py`

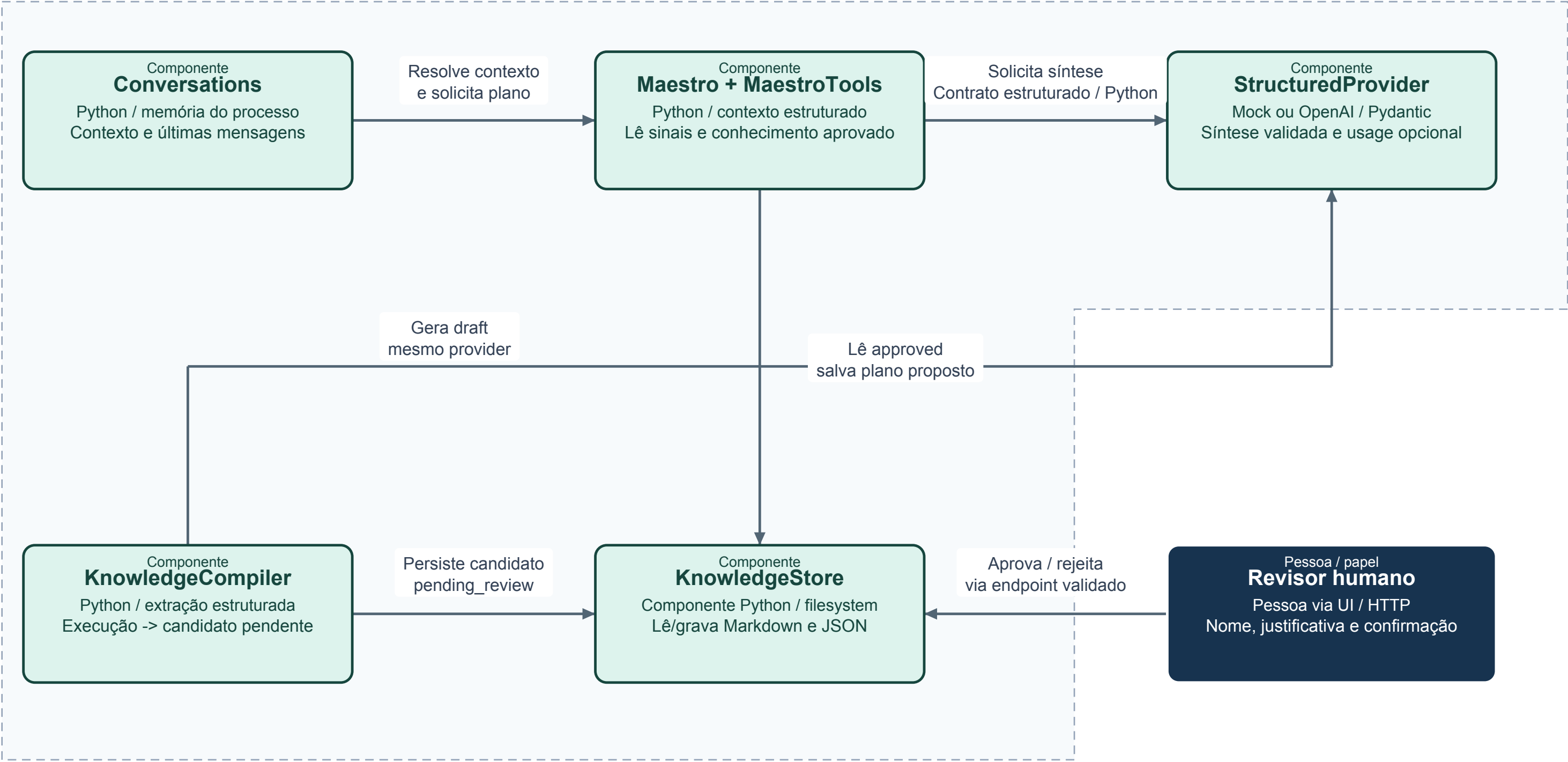
`src/control_tower/control_plane/interpretation.py`

`src/control_tower/control_plane/decision_engine.py`

`src/control_tower/cockpit/presentation.py`

Síntese, proposta e conhecimento

C3 · Componentes da API / gestão | Mesmo código reutilizado no processo MCP; Compiler/revisão acessados por HTTP



Leitura guiada | Síntese, proposta e conhecimento

Use depois do diagrama: explicar o porquê, não apenas os nomes das tecnologias.

COMO EXPLICAR

1. Maestro consulta sinais e memória aprovada, propõe plano e o persiste. Staffs são responsabilidades sugeridas; não são novos processos executados.
2. Compiler usa evidências de uma execução para produzir candidato. O servidor valida referências e estrutura; revisão humana avalia conteúdo.

LIMITES IMPORTANTES

1. O backend não tem autenticação/RBAC empresarial: o formulário é uma barreira didática de revisão, não comprovação inviolável de identidade.
2. Conhecimento aprovado melhora contexto do Maestro. Não há treinamento de pesos nem injeção automática no LangGraph do próximo incidente. Planos e candidatos não são autorização empresarial.

ONDE ENCONTRAR NO CÓDIGO

`src/control_tower/cockpit/conversation.py`

`src/control_tower/cockpit/assistance.py`

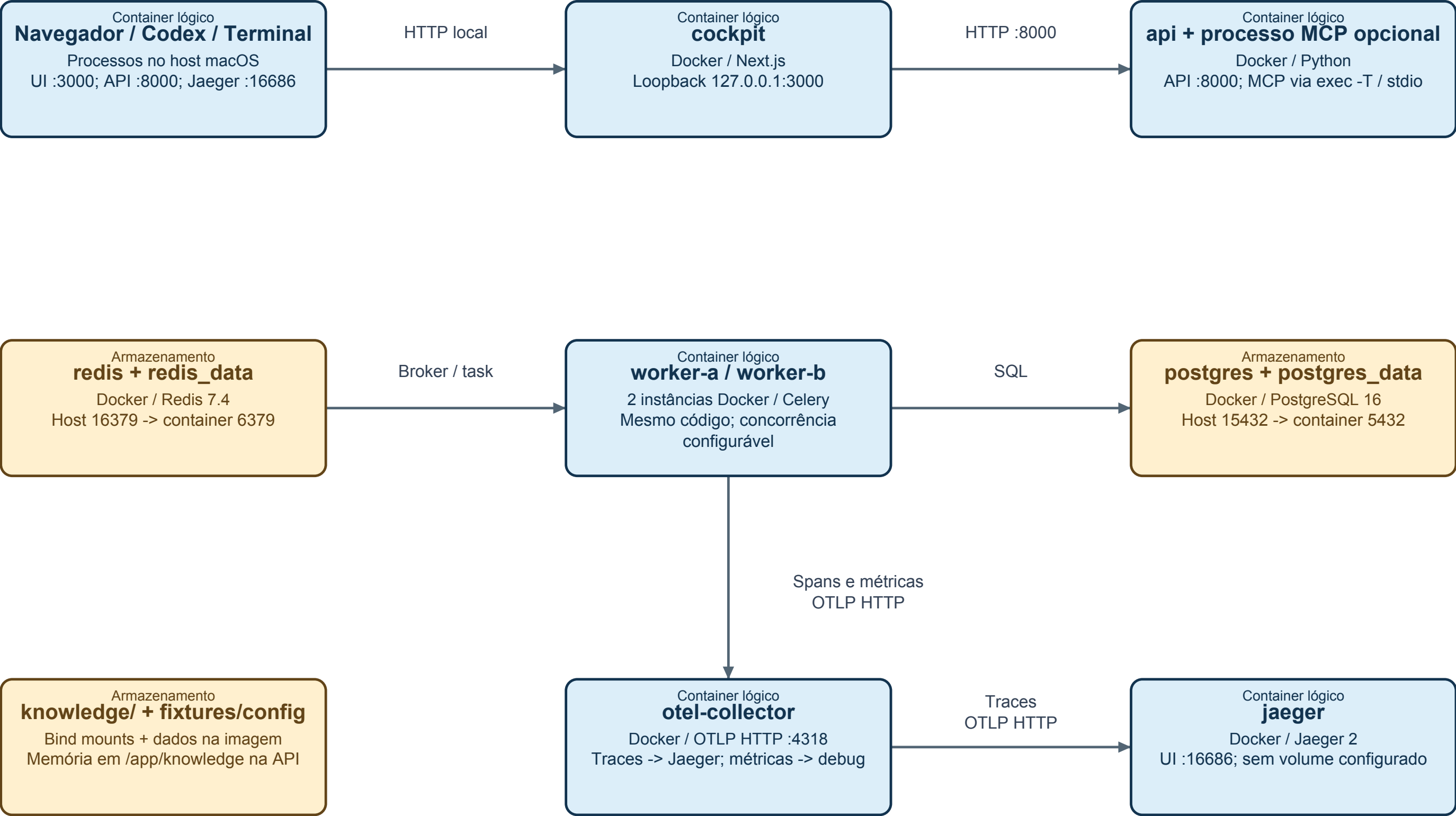
`src/control_tower/cockpit/llm.py`

`src/control_tower/cockpit/knowledge.py`

`src/control_tower/cockpit/api.py`

Como isso roda no computador do professor?

Complemento · Implantação local | Docker Compose; réplicas físicas não são novos tipos de container C4



Leitura guiada | Como isso roda no computador do professor?

Use depois do diagrama: explicar o porquê, não apenas os nomes das tecnologias.

COMO EXPLICAR

1. O desenho destaca instâncias físicas e portas. As demais dependências são as das vistas C2; aqui não repetimos todas as setas. API e MCP também exportam telemetria.
2. O launcher combina cinco arquivos Compose. O MCP usa a imagem, ambiente e mount de knowledge do container API; não abre um endpoint remoto.

LIMITES IMPORTANTES

1. Persistência: volumes para Redis/PostgreSQL; bind mount para conhecimento. Estado LangGraph e sessões de chat são memória de processo. Jaeger não tem volume persistente no Compose.
2. OpenAI é externa e acessada por HTTPS quando habilitada. Serviços do LAB ficam no localhost; não há Kubernetes, balanceador, alta disponibilidade nem autenticação empresarial nesta implantação.

ONDE ENCONTRAR NO CÓDIGO

`compose.yaml`

`compose.override.yaml`

`compose.lesson04.yaml`

`compose.lesson04-complete.yaml`

`compose.lesson04-cockpit.yaml`

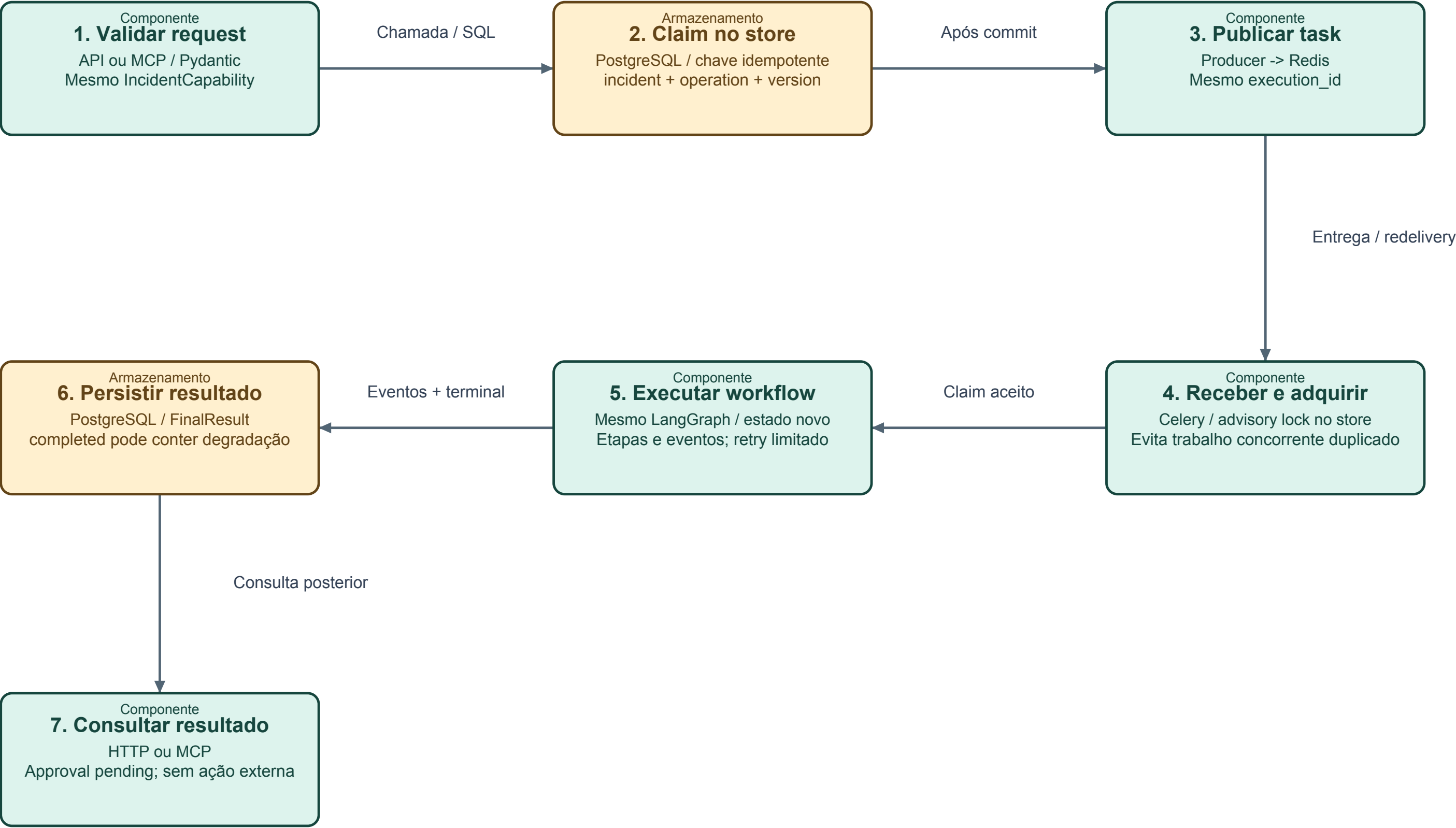
`scripts/cockpit.sh`

`scripts/start_maestro_mcp.sh`

`deploy/otel/collector.yaml`

Do pedido ao resultado durável

Complemento · Fluxo dinâmico | Ordem conceitual do caminho HTTP/MCP; at-least-once e idempotência



Leitura guiada | Do pedido ao resultado durável

Use depois do diagrama: explicar o porquê, não apenas os nomes das tecnologias.

COMO EXPLICAR

1. O cliente recebe um ID após submissão; consulta status/result separadamente. Workers podem receber a mesma identidade mais de uma vez.
2. Retry tenta novamente; redelivery recupera entrega após perda de worker. O advisory lock vive na sessão, sem manter uma transação aberta durante a inferência.

LIMITES IMPORTANTES

1. Idempotência no store não equivale a exactly-once para toda dependência externa. Uma falha antes de persistir conclusão pode repetir chamadas LLM e custo.
2. Falha transitória de LLM tem retry limitado e decisão explícita de fallback: degradação determinística apenas para `reference_replay` seguro, ou `human_review_required`. Não há fallback de produção genérico para mock.

ONDE ENCONTRAR NO CÓDIGO

`src/control_tower/distributed/producer.py`

`src/control_tower/distributed/store.py`

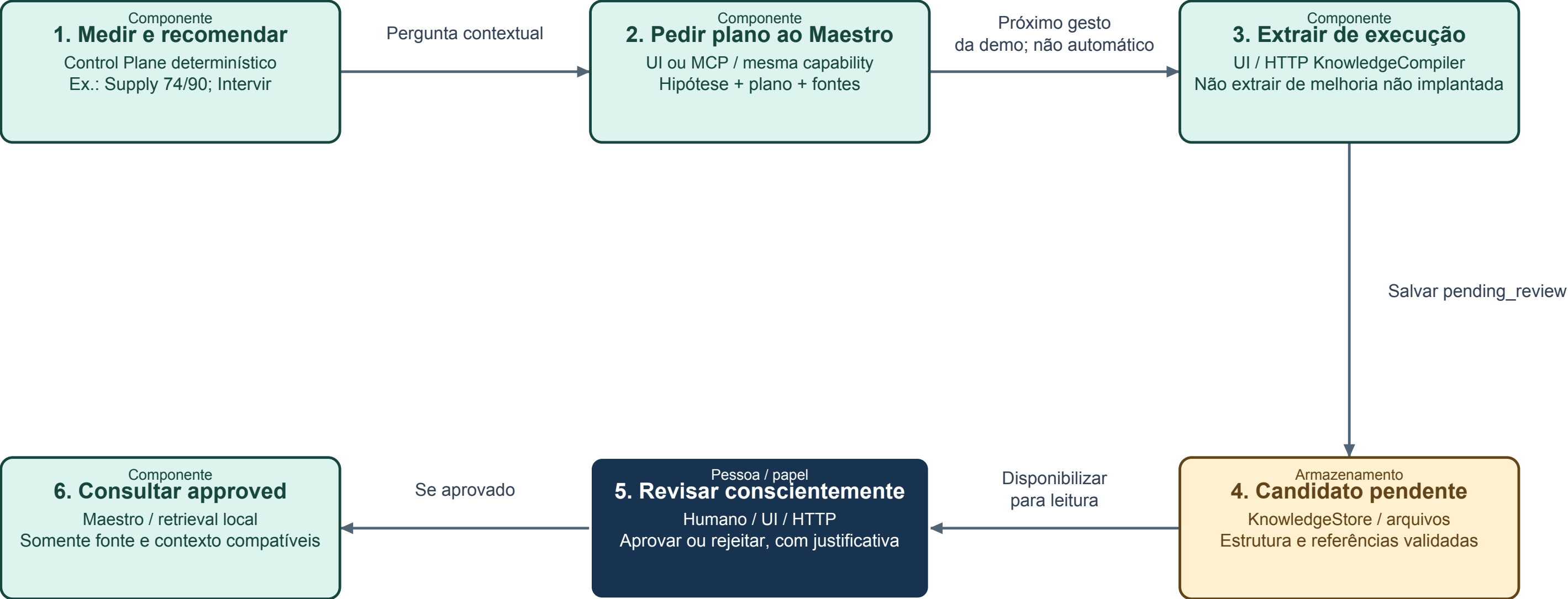
`src/control_tower/distributed/tasks.py`

`src/control_tower/distributed/llm_runtime.py`

`src/control_tower/distributed/durable.py`

Da proposta ao contexto reutilizável

Complemento · Fluxo dinâmico | Interação humana obrigatória; nenhum ACT empresarial



Leitura guiada | Da proposta ao contexto reutilizável

Use depois do diagrama: explicar o porquê, não apenas os nomes das tecnologias.

COMO EXPLICAR

1. A seta entre plano e extração é uma sequência de ações do professor, não uma automação. O plano não foi executado; a lição usa uma execução anterior disponível.
2. Aprovação de conhecimento altera o status do documento. Não aprova o incidente, não muda lifecycle e não executa o plano.

LIMITES IMPORTANTES

1. Retrieval usa documentos aprovados, filtrados por fonte e tags; é busca local limitada, não RAG vetorial.
2. Sessão HTTP e sessão MCP não são a mesma conversa. Um documento aprovado e um plano persistido podem ser lidos por ambos os processos.

ONDE ENCONTRAR NO CÓDIGO

`src/control_tower/cockpit/api.py`

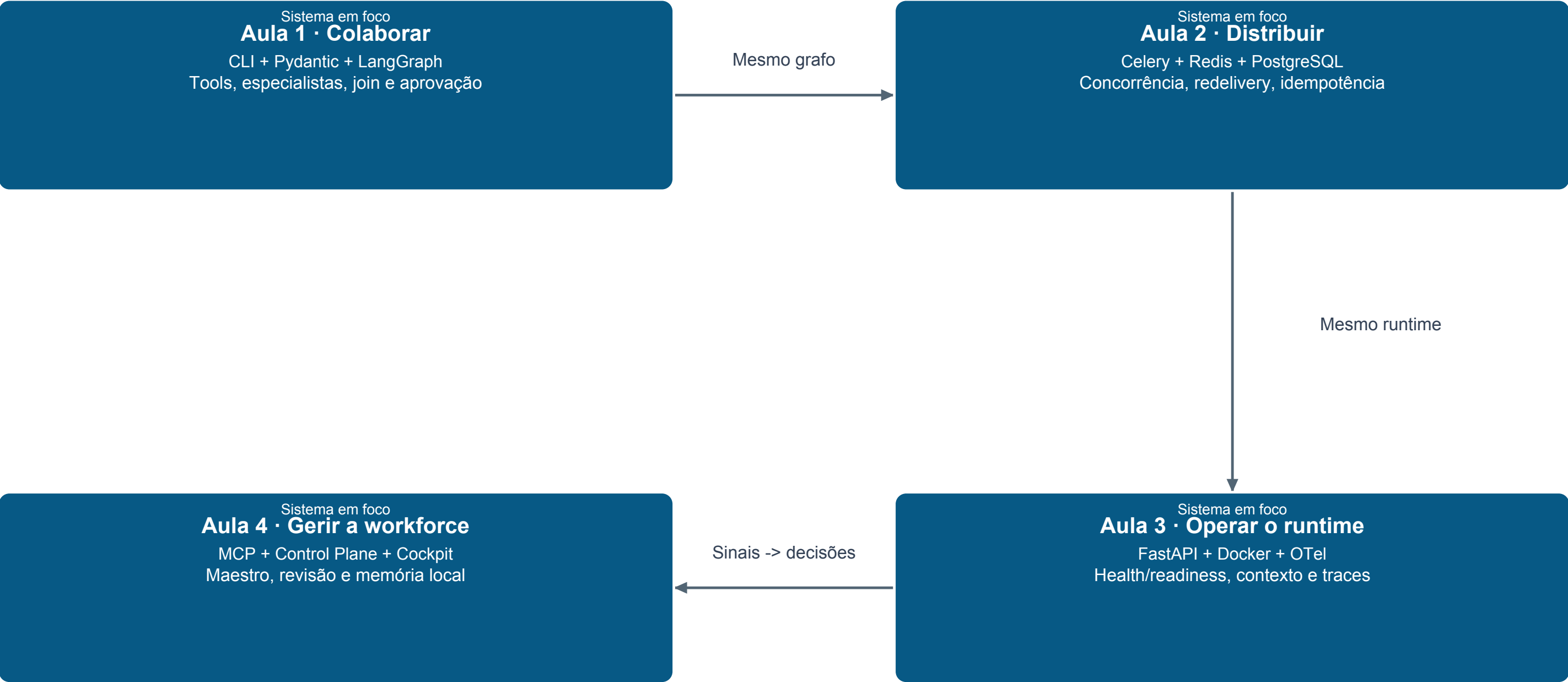
`src/control_tower/cockpit/assistance.py`

`src/control_tower/cockpit/conversation.py`

`src/control_tower/cockpit/knowledge.py`

Um sistema, quatro necessidades

Complemento · Evolução pedagógica | Camadas acumuladas; branches e tags preservam os estados didáticos



Leitura guiada | Um sistema, quatro necessidades

Use depois do diagrama: explicar o porquê, não apenas os nomes das tecnologias.

COMO EXPLICAR

1. Comece com colaboração local. A necessidade de concorrência leva à fila e aos workers; implantação e diagnóstico levam à API e à telemetria; gestão exige identidade, metas e memória.
2. As camadas não substituem o grafo por quatro aplicações. O caminho local continua útil para mock e demonstração conceitual.

LIMITES IMPORTANTES

1. PostgreSQL entra na Aula 2, não na Aula 3. MCP e Control Plane entram na Aula 4; OpenTelemetry já é da Aula 3.
2. Kubernetes, multi-provider routing, circuit breakers avançados, SLO-driven fallback e ACT autônomo não fazem parte da arquitetura implementada atual.

ONDE ENCONTRAR NO CÓDIGO

[docs/course/lesson-01-runbook.md](#)

[docs/course/lesson-02-runbook.md](#)

[docs/course/lesson-03-runbook.md](#)

[docs/course/lesson-04-final-runbook.md](#)

As três fronteiras que não podem se confundir

Medir, interpretar e autorizar são responsabilidades diferentes.

Código determinístico

Inventory, suppliers, production, logistics, cenário/custos, políticas, Finance, validação e Decision Engine permanecem em código. O LLM não substitui o banco, a tool ou a regra.

Interpretação por LLM

No workflow: Supervisor, sínteses dos especialistas, Challenger e Recommendation. Na gestão: Maestro e Knowledge Compiler. Outputs têm contratos Pydantic; referências válidas não garantem verdade semântica.

Autorização humana

Human approval do incidente é um estado terminal pendente. Revisão do conhecimento é uma operação implementada na UI/API. Aprovar uma lição não aprova compra, transferência, plano ou mudança de lifecycle.

O que ficou fora

Sem ACT, auto-deploy, automodificação, IAM empresarial, multi-tenancy, RAG vetorial, checkpoint por nó, transactional outbox ou garantia exactly-once de chamadas externas. A2A não foi implementado; MCP é local stdio.

Referências e manutenção

Arquitetura documentada a partir do código e da configuração, não de uma stack desejada.

Modelo C4 - contexto | <https://c4model.com/diagrams/system-context>

Modelo C4 - containers | <https://c4model.com/diagrams/container>

Modelo C4 - componentes | <https://c4model.com/diagrams/component>

Convenção desta entrega

As vistas C1-C3 usam notação C4 simplificada com tipo, responsabilidade e tecnologia em cada caixa. Mermaid flowchart é usado por compatibilidade com a renderização do GitHub. SVG e PDF preservam um layout explícito; a disposição automática do Mermaid pode variar.

Fontes e reprodução

docs/architecture/c4/model.json é a fonte editável. build.py gera diagramas .mmd, .svg e este PDF com ReportLab, sem executar o sistema ou chamar providers. README.md reúne o catálogo, o vínculo com os arquivos reais e as instruções de atualização.

Não foi necessário reexecutar workloads nem alterar código do produto. As limitações operacionais foram conferidas nos módulos de execução, gestão e Compose. A arquitetura é do LAB local; sua promoção a produção requer outro projeto de implantação e segurança.